



SUBMISSION OF TASK:

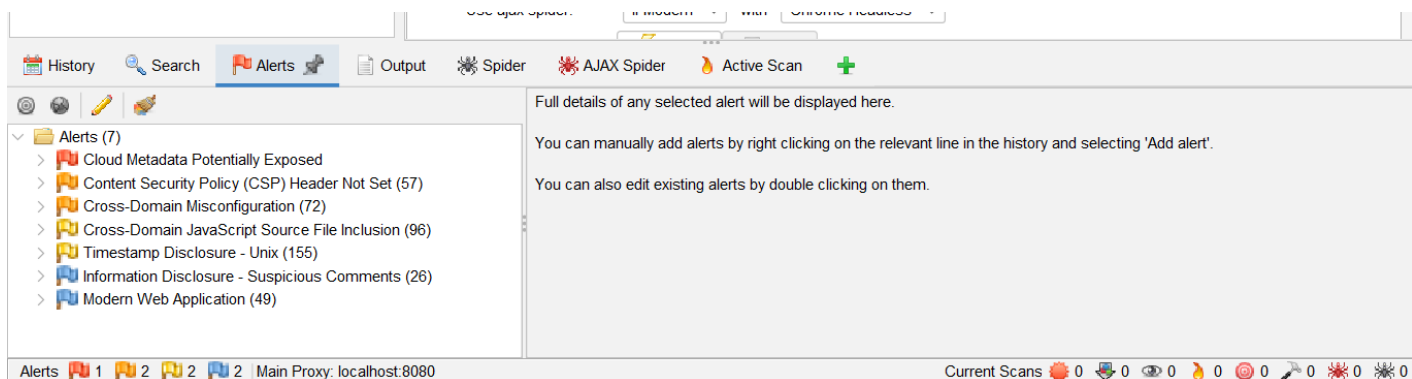
NAME: ADEPOJU ESTHER ITUNUOLUWA

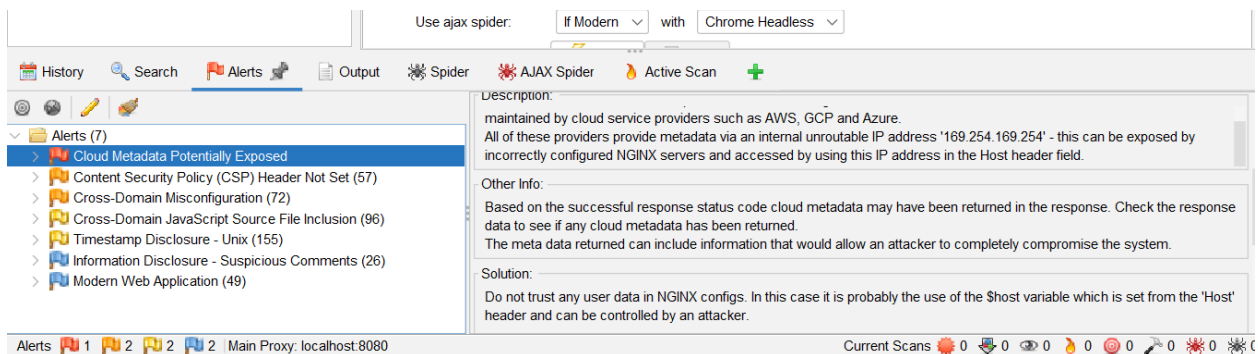
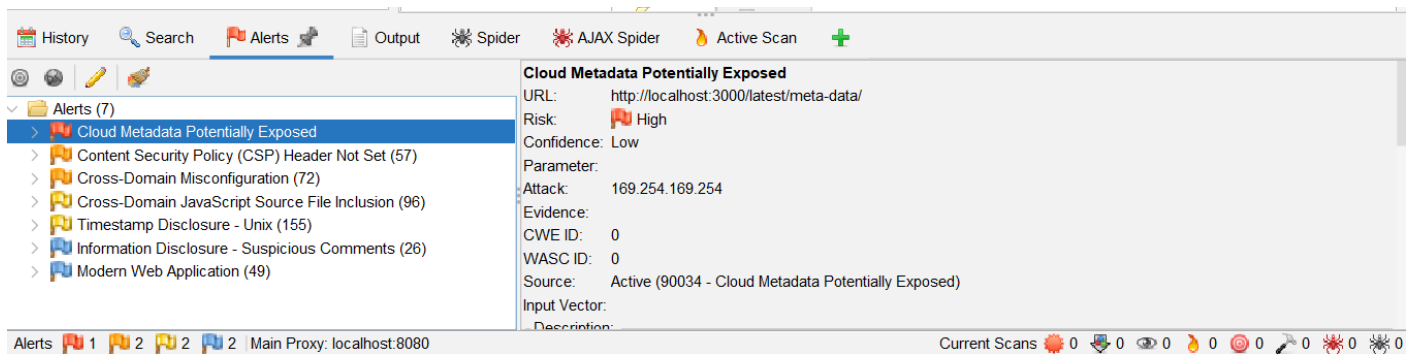
TASK: WEB APPLICATION SECURITY TESTING

TOOLS USED: OWASP JUICE SHOP, DOCKER DESKTOP OWASP ZAP, BURP SUITE, MS WORD (for reporting).

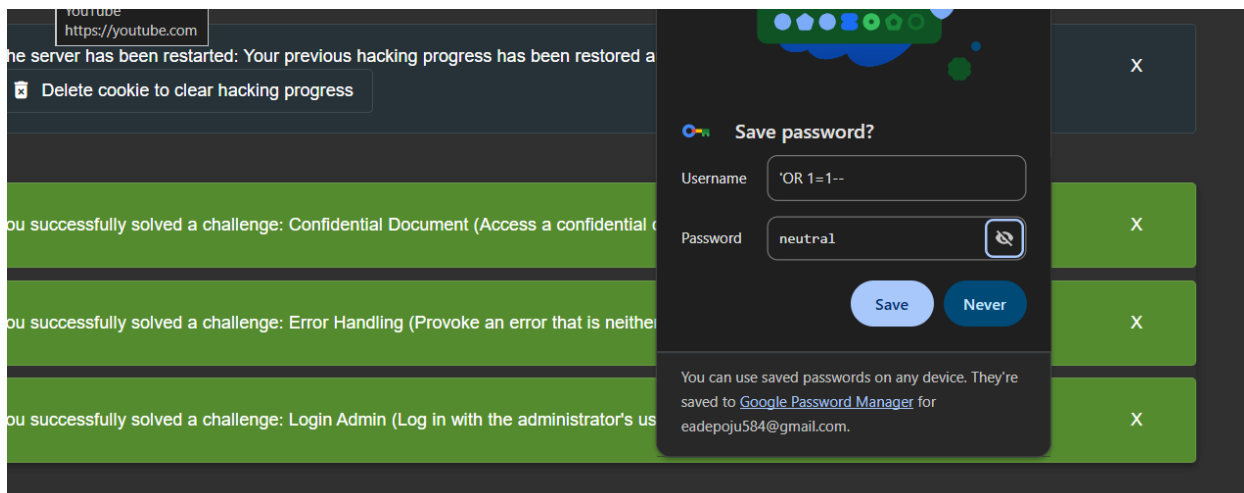
DETAILS: In this task, I performed a vulnerability assessment of an open source web application (OWASP Juice Shop in this case). I was able to successfully identify security flaws in the application using OWASP ZAP.

After attacking and identifying the vulnerabilities, the following were what I detected, mapped to OWASP Top 10 vulnerabilities, and the solutions to them.





Zap scan report on OWASP Juice Shop



PROOF OF SQL INJECTION USING ADMIN PRIVILEGE

VULNERABILITY 1: Cloud Metadata Potentially Exposed.

MAPPING TO OWASP TOP 10: **OWASP TOP 10: A03:2021 – Injection.**

IMPACT LEVEL: **HIGH**

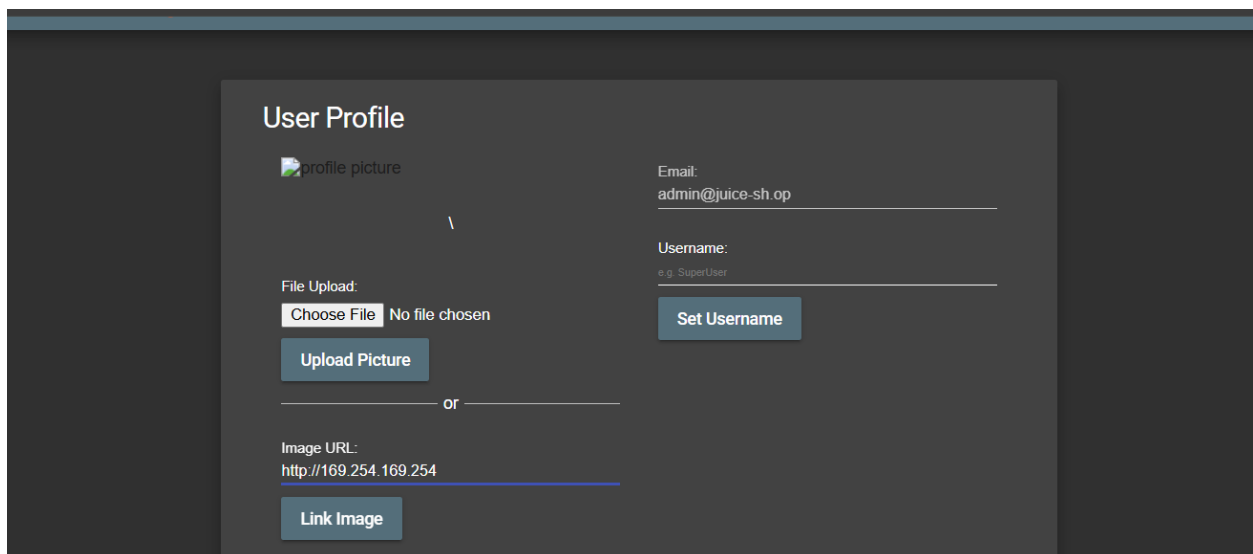
My Findings: It was discovered that the cloud metadata attack attempts to exploit a misconfigured NGINX server to access the instance metadata maintained by cloud service providers.

IMPACT ON THE WEBSITE:

It exposes the cloud instance metadata, including IAM roles and temporary credentials. There is also potential for privilege escalation in the cloud environment, along with data exfiltration.

Remediation steps:

1. Do not trust any user data in NGINX configs. In this case, it is probably the use of the \$host variable, which is set from the 'Host' header and can be controlled by an attacker.
2. Validate and sanitize all URLs used in outbound requests. Do not allow user-supplied or dynamic URLs unless strictly necessary.
3. Implement allow-listing for outbound requests (permit only trusted domains).



The screenshot shows a 'User Profile' form with the following elements:

- Profile Picture:** A placeholder icon labeled 'profile picture'.
- Email:** A text input field containing 'admin@juice-sh.op'.
- Username:** A text input field containing 'e.g. SuperUser'.
- File Upload:** A section with a 'Choose File' button, the text 'No file chosen', and an 'Upload Picture' button.
- Image URL:** A text input field containing 'http://169.254.169.254'.
- Buttons:** A 'Set Username' button next to the username field and a 'Link Image' button below the image URL field.

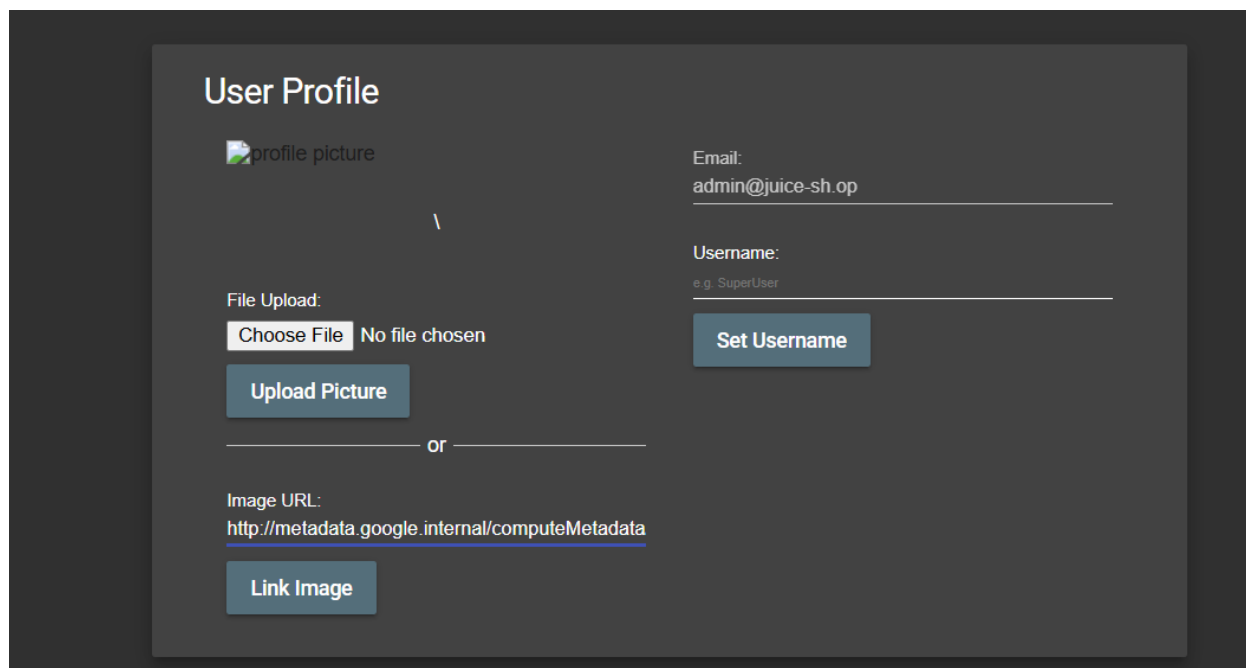


Figure 1-2: Screenshot of cloud meta data vulnerability.

VULNERABILITY 2: **Content Security Policy (CSP) Header Was Not Set**

MAPPING TO OWASP TOP 10: **A10 — Server-Side Request Forgery (SSRF).**

IMPACT LEVEL: **MEDIUM**

My Findings: It was discovered that Content Security Policy which is an added layer of security that helps to detect and mitigate certain types of attacks, including XSS, was not set. This was used to distribute malware eventually.

IMPACT ON THE WEBSITE:

There was increased exposure to XSS, injection attacks, and malicious third-party content; also, attacker-controlled scripts can steal cookies, tokens, or sensitive data. There is also a risk of client-side compromise, reputation damage, or regulatory non-compliance.

Remediation steps:

1. Ensure that the web server, application server, load balancer, etc., is configured to set the Content-Security-Policy header.
2. Ensure only trusted domains (your own and vetted CDNs) are included.
3. Avoid use of unsafe-inline directives as they weaken protection.
4. Test your policy in Report-Only mode first to detect violations without blocking content.

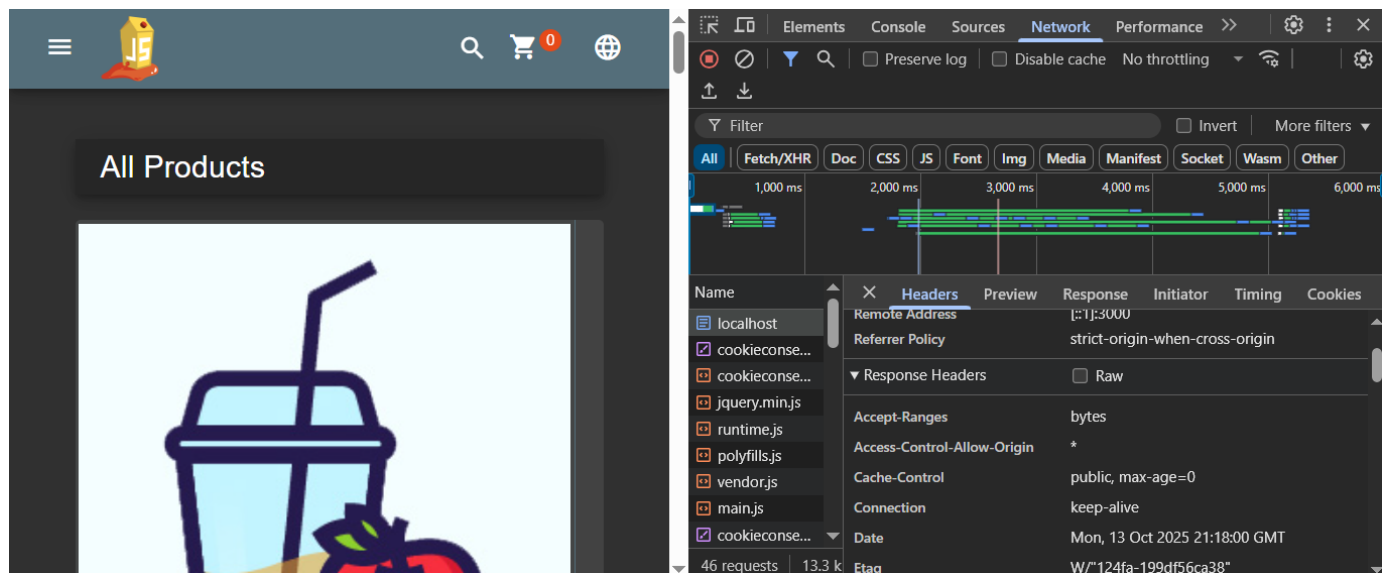


Figure 3: Content security policy header was not set under RESPONSE header in the developer's tool.

VULNERABILITY 3: Cross-Domain Misconfiguration

MAPPING TO OWASP TOP 10: A05: Security Misconfiguration

IMPACT LEVEL: **MEDIUM**

My Findings: It was discovered that though web browser data loading may be possible, due to a Cross Origin Resource Sharing misconfiguration on the web server, CORS controls which external domains can actually make requests to your web application using a browser, which allows malicious external websites to read sensitive data.

1. Attackers would be able to read API responses (e.g., GET /rest/user/whoami, GET /api/orders) if Access-Control-Allow-Origin is set too permissively and Access-Control-Allow-Credentials: true is enabled.

2. With credentials (session cookies or tokens automatically sent by the browser when credentials: include is used), attackers can trigger state-changing requests (change password, place orders, add or remove items, use stored payment info) depending on the endpoints accessible.

Remediation steps:

1. Ensure that sensitive data is not available in an unauthenticated manner (using IP address white-listing, for instance).

2. Configure the "Access-Control-Allow-Origin" HTTP header to a more restrictive set of domains, or remove all CORS headers entirely, to allow the web browser to enforce the Same Origin Policy (SOP) in a more restrictive manner.

3. Use a strict allow list of origins — only permit known, trusted domains (e.g., your official frontend domain).

Burp Suite Community Edition v2025.8.8 - Reproxy Proxy

Dashboard Target **Proxy** Intruder Repeater View Help

Intercept HTTP history WebSockets history Match and replace Proxy settings

Filter settings: Hiding CSS, image and general binary content

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies
12	http://localhost:3000	GET	/										127.0.0.1	
14	http://localhost:3000	GET	/										127.0.0.1	
16	http://localhost:3000	GET	/										127.0.0.1	
17	http://localhost:3000	GET	/										127.0.0.1	
18	http://localhost:3000	GET	/			200	75471	HTML		OWASP Juice Shop			127.0.0.1	
19	http://localhost:3000	GET	/			200	75471	HTML		OWASP Juice Shop			127.0.0.1	
20	http://localhost:3000	GET	/runtime.js			200	3818	script	js				127.0.0.1	
21	http://localhost:3000	GET	/main.js			200	454702	script	js				127.0.0.1	
22	http://localhost:3000	GET	/vendor.js			200	1689518	script	js				127.0.0.1	
23	http://localhost:3000	GET	/polyfills.js			200	35325	script	js				127.0.0.1	
24	http://cdnjs.cloudflare.com	GET	/ajax/libs/cookieconsent/2.3.1.0/co...			200	21764	script	js				104.17.24.14	
26	http://cdnjs.cloudflare.com	GET	/ajax/libs/jquery/2.2.4/jquery.min.js			200	86539	script	js				104.17.24.14	
27	http://localhost:3000	GET	/runtime.js			304	391	script	js				127.0.0.1	
28	http://localhost:3000	GET	/main.js			304	393	script	js				127.0.0.1	
29	http://localhost:3000	GET	/polyfills.js			304	392	script	js				127.0.0.1	
31	http://localhost:3000	GET	/vendor.js			304	394	script	js				127.0.0.1	
33	http://localhost:3000	GET	/socket.io/?EIO=4&transport=poll...	✓				io/					127.0.0.1	
34	http://localhost:3000	GET	/assets/i18n/en.json					script	json				127.0.0.1	
35	http://localhost:3000	GET	/rest/admin/application-configurat...										127.0.0.1	
36	http://localhost:3000	GET	/rest/admin/application-version										127.0.0.1	
37	http://localhost:3000	GET	/socket.io/?EIO=4&transport=poll...	✓				io/					127.0.0.1	
38	http://localhost:3000	GET	/api/Challenges/?name=Score%20...	✓									127.0.0.1	

Request Inspector

Pretty Raw Hex

Event log (29) All issues

Memory: 153.7MB Disabled

Figure 4: Screenshot of the intercepted data on Burp suite.

VULNERABILITY 4: Cross-Domain JavaScript Source File Inclusion

MAPPING TO OWASP TOP 10: A08:2021 – Software and Data Integrity Failures

IMPACT LEVEL: LOW

My Findings: It was discovered that the page includes one or more script files from a third-party domain, and this causes the browser to automatically download and execute a file with the same privileges as scripts from a legitimate website.

IMPACT ON THE WEBSITE:

A malicious script can read session cookies or JWT tokens, and form inputs, PII, and payment details can be sent to attacker servers. An attacker can change page content, insert ads, or phishing forms.

Remediation steps:

1. Manually confirm that the timestamp data is not sensitive, and that the data cannot be aggregated to disclose exploitable patterns.
2. When including third-party scripts from CDNs, use SRI to force the browser to check the script hash before executing.
3. Only load scripts over HTTPS and enable HSTS to ensure integrity and confidentiality of third-party delivery.

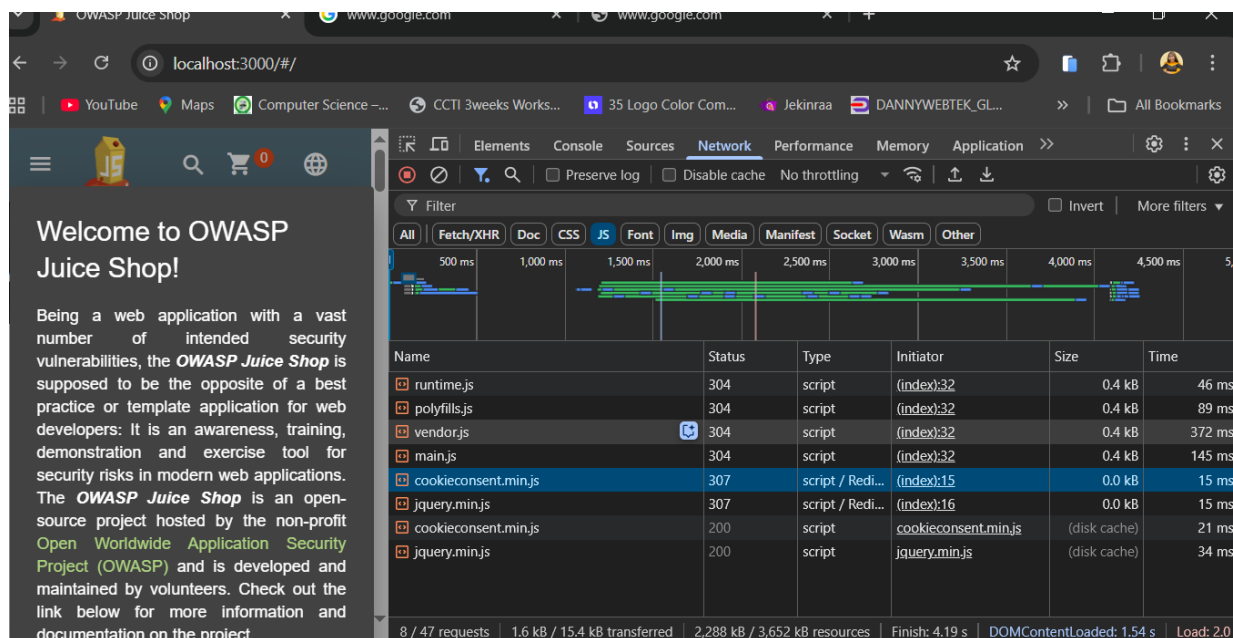


Figure 5: Screenshot of the external JavaScript embedded in the server.

VULNERABILITY 5: Timestamp Disclosure – Unix

MAPPING TO OWASP TOP 10: A05:2021 – Security Misconfiguration

IMPACT LEVEL: **LOW**

My Findings: It was discovered that a timestamp was disclosed by the application/web server – Unix in the HTTP header section.

IMPACT ON THE WEBSITE:

1. Attackers would be able to know when the website or server was updated and also get to know about the internal structure or version.
2. Attackers can use this to perform a reconnaissance attack by knowing when the last software or files were updated. Attackers can also see recent updates and know exactly what was being changed, and focus on related areas.

Remediation steps:

1. Manually confirm that the timestamp data is not sensitive, and that the data cannot be aggregated to disclose exploitable patterns.
2. Disable 'Last Modified' headers on sensitive responses.

SKILLS GAINED:

1. Web application vulnerability assessment
2. Knowledge of the top OWASP top 10 threats and mapping to website vulnerabilities
3. Security documentation and reporting.
4. Basic ethical hacking and penetration testing.